

Qryptalk
A Post Quantum Cryptography Messaging App

Project Abstract

B. Tech 4th Year | 1st Semester

Submitted by

V. Premnath

R200347

under the guidance of

Mr. P. Santhosh Kumar

ASSISTANT PROFESSOR



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES

RK VALLEY, Vempalli (M), Kadapa (Dist),

Andhra Pradesh(S), 516330

A Proof-of-Concept Quantum-Safe Messaging Application using CRYSTALS-Kyber

1. Introduction & Motivation

The impending arrival of large-scale quantum computers poses a significant threat to modern digital security. Conventional public-key cryptosystems, such as RSA and Elliptic Curve Cryptography (ECC), which underpin the security of nearly all current communication platforms, are vulnerable to attacks by quantum algorithms like Shor's algorithm. This vulnerability creates the risk of "harvest now, decrypt later" attacks, where encrypted data is collected today to be decrypted once quantum computers become available. To counter this threat, the field of Post-Quantum Cryptography (PQC) has emerged, focusing on developing new cryptographic algorithms that are secure against both classical and quantum computers. This project presents a proof-of-concept (PoC) Quantum-Safe Messaging App designed to demonstrate the practical application of PQC in securing end-to-end communication.

2. Project Objective

The primary objective of this project is to design, develop, and implement a functional messaging application that uses a standardized PQC algorithm for its key exchange protocol. This application will serve as a practical demonstration of how next-generation cryptographic standards can be integrated into real-world software to build communication systems resilient to future quantum threats. The project emphasizes a clean, modular architecture and clear documentation to serve as an educational resource for students and developers interested in applied cryptography and secure software development.

3. Technical Architecture & Implementation

The application is architected as a client-to-client messaging system that ensures confidentiality through a hybrid encryption scheme.

- **Post-Quantum Key Exchange:** The core of the application's security lies in its key exchange mechanism. We will implement **CRYSTALS-Kyber**, a lattice-based Key Encapsulation Mechanism (KEM) chosen by the U.S. National Institute of Standards and Technology (NIST) as a primary standard for PQC. The key exchange process involves:
 1. The recipient generating a public/private key pair based on the Kyber algorithm.
 2. The sender using the recipient's public key to generate a shared secret and an encrypted ciphertext (the "encapsulation").
 3. The recipient using their private key to "decapsulate" the ciphertext and retrieve the identical shared secret.
- **Symmetric Message Encryption:** The shared secret derived from the Kyber exchange is not used to encrypt messages directly. Instead, it serves as a key for a well-established, high-performance symmetric-key algorithm, such as **AES-256 in Galois/Counter Mode (GCM)**. This hybrid approach combines the quantum resistance of Kyber for the key exchange with the efficiency of AES for encrypting the bulk message data, which is a standard practice in modern cryptography.

- **User Interface (UI):** A minimalist and intuitive graphical user interface will be developed to facilitate user interaction. The UI will allow users to manage contacts (based on their public keys), send encrypted messages, and view decrypted conversations in a clean, readable format.

4. Features & Deliverables

- A fully functional desktop messaging application (PoC).
- End-to-end encryption secured with the Kyber PQC algorithm.
- A simple and clear user interface for demonstrating core functionality.
- Source code with comprehensive comments and documentation explaining the cryptographic operations and software architecture.

5. Tech Stack

This stack is chosen for its modern approach to cross-platform desktop development, robust cryptographic support through WebAssembly, and rapid prototyping capabilities.

- **Application Framework:**
 - **Electron:** A framework for building cross-platform (Windows, macOS, Linux) desktop applications using web technologies. It's ideal for creating a rich graphical UI for the messaging app.
- **Frontend (User Interface):**
 - **Language: TypeScript.** Provides static typing over JavaScript, reducing errors and improving code quality, which is crucial for a security-focused application.
 - **UI Library: React.js.** A powerful and popular library for building modern, component-based user interfaces. It works seamlessly within Electron.
 - **Styling:** Standard CSS or a framework like **Tailwind CSS** for rapid UI development.
- **Backend (Relay Server):**
 - **Runtime: Node.js.** Uses the same JavaScript/TypeScript language as the frontend, simplifying the development process.
 - **Framework: Express.js.** A minimalist web framework for creating the server logic.
 - **Real-time Communication: WebSockets** (using the ws library). Essential for the instant, bidirectional communication required for a chat application. The server will act as a simple relay for messages and public keys.
- **Cryptography Libraries :**
 - **Post-Quantum Cryptography (Kyber): OQS.js.** This is the JavaScript/WebAssembly version of the Open Quantum Safe (OQS) liboqs project. It provides pre-compiled, high-performance implementations of CRYSTALS-Kyber and other NIST PQC candidates. This is the core of the quantum-safe key exchange.
 - **Symmetric Encryption (AES-GCM): Web Crypto API (crypto.subtle).** This is the browser-native (and Electron-native) API for performing standard cryptographic operations. It is the most secure way to implement AES-256-GCM as the operations

are performed in a secure, optimized environment, protecting against many side-channel attacks.

- **Client-Side Storage:**

- **Message History & Contacts: SQLite.** A lightweight, file-based SQL database perfect for storing chat logs and user contacts locally. The better-sqlite3 library is a great choice for this in a Node.js/Electron environment.
- **Configuration & Keys: electron-store.** A simple library for managing user settings. For the PoC, it can store the user's private key in an encrypted file. *Note: A production-grade app would use the operating system's native keychain for more secure private key storage.*

6.Core Cryptographic Workflow (Explained)

This is the step-by-step process of how two users, Alice and Bob, would establish a secure channel and exchange messages.

1. Initial Key Generation:

- **When Alice and Bob first run the app, the client uses the OQS.js library to generate a long-term CRYSTALS-Kyber key pair for each of them (a public key pk and a private key sk).**
- **Their public keys are uploaded to the central relay server, associated with their usernames.**

2. Session Initiation (Alice wants to message Bob):

- **Alice's app requests Bob's public key (pk_B) from the relay server.**
- **Alice's app now has its own key pair (pk_A, sk_A) and Bob's public key (pk_B).**

3. Quantum-Safe Key Exchange (Kyber KEM):

- **Encapsulation (Alice's side): Alice's client uses Bob's public key (pk_B) to perform the Kyber "encapsulation" operation. This generates two things:**
 1. **A shared secret** (a fresh, random 256-bit key for symmetric encryption, let's call it ss_AB).
 2. **A ciphertext** (let's call it ct_B), which is the encapsulated form of the shared secret.
- **Alice sends the ct_B to Bob via the relay server. The shared secret ss_AB never leaves her device.**

4. Session Establishment (Bob's side):

- **Decapsulation (Bob's side): Bob's client receives ct_B. It uses its own private key (sk_B) and the received ct_B to perform the Kyber "decapsulation" operation.**

- If successful, this operation reveals the exact same shared secret `ss_AB` on Bob's side. An attacker who only sees `ct_B` and Bob's public key cannot derive the secret, even with a quantum computer.

5. Symmetric Message Encryption:

- Now both Alice and Bob possess the identical, secure shared secret `ss_AB`.
- When Alice wants to send a message ("Hello Bob"), her client encrypts it using AES-256-GCM with `ss_AB` as the key.
- The encrypted message is sent to Bob. Bob's client uses its copy of `ss_AB` to decrypt it with AES-256-GCM. This process is extremely fast and efficient for ongoing conversation.

7. System Architecture & Data Flow

The system will use a centralized relay server architecture, which is simple and effective for a proof-of-concept.

- **Clients:** The Electron desktop applications running on users' computers. They handle all cryptographic operations, ensuring end-to-end encryption. The private keys never leave the client.
- **Relay Server:** A simple Node.js server. Its only jobs are:
 1. **User Registration:** Storing usernames and their associated public Kyber keys.
 2. **Message Relaying:** Receiving encrypted messages from one client and forwarding them to the correct recipient client via WebSockets. The server cannot read the messages.
 3. **Presence:** Indicating which users are currently online.

Data Flow Example: Alice's Client -> [Message Encrypted with AES(`ss_AB`)] -> WebSocket Server -> [Message Relayed] -> Bob's Client -> [Message Decrypted with AES(`ss_AB`)]

8. Significance

This project provides a tangible and accessible implementation of a leading PQC standard, bridging the gap between theoretical cryptography and practical application.

Key Exchange Is More Vulnerable

1. **RSA & Diffie-Hellman**
 - Currently, most secure communications (TLS, VPNs, etc.) use **RSA** or **Diffie-Hellman (DH/ECDH)** for key exchange.
 - A sufficiently powerful quantum computer running **Shor's Algorithm** can **factor RSA moduli** or **solve discrete logs** in polynomial time → breaking RSA, DH, and ECC.
2. **Symmetric Encryption** (AES, ChaCha20, etc.)

- These are relatively safer.
- Quantum attacks (like Grover's Algorithm) only give a **quadratic speedup** → meaning AES-256 is still safe (effective ~AES-128 under Grover).

3. Digital Signatures

- RSA and ECC signatures are also vulnerable to Shor's algorithm.
- But their usage frequency (per session or per transaction) is much less compared to how often key exchanges happen on the internet (billions of TLS handshakes daily).

4. Key Exchange = First Doorway

- If an attacker breaks the **session key derivation**, they can decrypt all past and future communications.
- This is especially dangerous in the “**store now, decrypt later**” attack model (where attackers record encrypted traffic today and wait until quantum computers can break it).

That's why **NIST's PQC standardization (2022)** prioritized **KEMs (like Kyber)** first — since securing **key exchange** is more urgent than even post-quantum signatures.

Risk Ranking (Quantum Threats in Cryptography)

1. Key Exchange (Most Vulnerable)

- Classical protocols like **RSA**, **Diffie–Hellman**, and **Elliptic Curve Diffie–Hellman (ECDH)** rely on **integer factorization** or **discrete logarithm** problems.
- **Shor's Algorithm** breaks them in **polynomial time** → making all classical public-key exchanges insecure.
- **Risk Level: Extremely High**

2. Digital Signatures (Highly Vulnerable)

- RSA signatures, DSA, ECDSA → **all breakable by Shor's algorithm**.
- This threatens **authentication, certificates, and trust chains (PKI)**.
- **Risk Level: High**

3. Symmetric Encryption (Least Vulnerable)

- AES and other symmetric ciphers are **not directly broken** by quantum computers.
- Grover's Algorithm only gives a **quadratic speed-up** → AES-128 security drops to 64-bit strength, but AES-256 remains strong.
- **Risk Level: Low**

Key Exchange > Signatures > Symmetric Encryption in terms of risk.

9. Future Extensions

- **Mobile Platform Integration:** Adapting the application for Android or iOS to explore PQC performance on mobile devices.
- **Hybrid Cryptosystem:** Implementing a dual key exchange that uses both a classical (e.g., ECC) and a quantum-safe (Kyber) algorithm simultaneously, ensuring security even if one is compromised.
- **Federated Architecture:** Expanding the system to support a decentralized or federated server model for managing user identities and public keys.

By demonstrating a working model of a quantum-safe communication tool, this project aims to highlight the feasibility of transitioning to PQC and to inspire further innovation in building secure, future-proof systems.